

Dart: 基本語法(2)

撰寫非同步程式碼

- 準備工作：
 - 建立一新目錄 dartex2
 - 利用 dart 指令，在 dartex2 建立一個 httpex 專案
 - 將 dartex1/cli/bin/cli.dart 程式內容，完整複製一份，完全覆蓋至 dartex2/httpex/bin/httpex.dart 檔案。
- Task1: 新增 http 軟體套件到專案裡
 - 打開 dartex2/httpex/pubspec.yaml 檔案
 - pubspec 檔案專門用來管理 Dart 專案屬性資料與包含在內的相關軟體套件。
 - 找到 dependencies 段落
 - 增加下列內容：

```
dependencies:  
  path: ^1.9.0  
  http: ^1.3.0
```

- 記得存檔
- 在 Terminal 文字介面視窗，於 httpex 目錄下，執行下列指令：

```
dart pub get
```

- 應該會看到下列訊息內容：

```
Resolving dependencies...  
Downloading packages...  
+ http 1.4.0  
  lints 5.1.1 (6.0.0 available)  
Changed 1 dependency!  
1 package has newer versions incompatible with dependency constraints.  
Try `dart pub outdated` for more information.
```

- Task2: 匯入 http 軟體套件
 - 打開 dartex2/httpex/bin/httpex.dart 檔案
 - 於檔案開頭，加入下列兩行程式：

```
import 'dart:io';  
import 'package:http/http.dart' as http;  
(以下省略...)
```

- as：表示將引入的軟體套件，以別名稱呼，方便後續程式的撰寫與呼叫，避免撞名。
- Task3: 執行 getWikipediaArticle 函式功能
 - 在 main() 函式之下，於 printUsage() 函式之後，加入 getWikipediaArticle() 函式：

[Back to top](#)

```
// 之前是你的 printUsage() 函式
(以上省略...)
```

```
Future<String> getWikipediaArticle(String articleTitle) async {
  // 這裡將來會增加很多程式
}
```

- Future 值返回值型態，是表示將來該函式執行完成後，將會產生 String 型態資料。
 - 但不是立即的產生，因為是非同步操作。
 - async 關鍵字標示本函式是非同步操作，允許在程式主體使用 await 修飾詞。
- 在 getWikipediaArticle() 函式內，建構 Uri 物件，代表 Wikipedia API 端點，可取得文章的總結。
 - getWikipediaArticle() 函式程式主體內容如下：

```
(以上省略...)
```

```
Future<String> getWikipediaArticle(String articleTitle) async {
  final url = Uri.https(
    'en.wikipedia.org', // Wikipedia API 網域主機名稱
    '/api/rest_v1/page/summary/$articleTitle', // 文章總結內容的 API 路
  );
  // (以下省略...)
}
```

- 使用 http 套件內的 get() 函式，製作一個 HTTP GET 的連線請求：

```
(以上省略...)
```

```
Future<String> getWikipediaArticle(String articleTitle) async {
  final url = Uri.https(
    'en.wikipedia.org',
    '/api/rest_v1/page/summary/$articleTitle',
  );
  final response = await http.get(url); // 製作 HTTP 連線請求

  if (response.statusCode == 200) {
    return response.body; // 假如連線成功，回傳 response 主體內容
  }

  // 假如連線失敗，回傳失敗訊息
  return 'Error: Failed to fetch article "$articleTitle". Status code: ${r
}
```

- await 修飾詞可暫停 getWikipediaArticle() 函式功能，直到 get 呼叫完成，並取回 http.Response 物件。
- response.statusCode 確認連線的要求是否成功。回應值如是 200，表示成功。
- 假如連線成功，回傳 response.body 內容。
- Task4: 整合 API 進 searchWikipedia() 函式內
 - 更新 searchWikipedia() 函式，將其定義成 async 非同步函式。程式內容如下：

```
(以上省略...)
void searchWikipedia(List<String>? arguments) async { // 此處增加 'async'
  final String? articleTitle;

  (以下省略...)

```

- 在此函式名稱後，宣告 `async` 是為了配合呼叫 `getWikipediaArticle()` 函式，預先做的準備。
- 於 `searchWikipedia()` 函式內，在 `stdin.readLineSync()` 函式取回字串時，處理空值或空字串問題：

```
(以上省略...)
void searchWikipedia(List<String>? arguments) async {
  final String articleTitle;

  if (arguments == null || arguments.isEmpty) {
    print('Please provide an article title.');
```

// 讀取輸入值

```
    final inputFromStdin = stdin.readLineSync();

    // 處理空值或空字串問題
    if (inputFromStdin == null || inputFromStdin.isEmpty) {
      print('No article title provided. Exiting.');
```

// 如果有不合理的輸入，離開此函式

```
      return;
    }
    articleTitle = inputFromStdin;
  } else {
    articleTitle = arguments.join(' ');
  }
  (以下省略...)

```

- 於 `searchWikipedia()` 函式內，修改最後三行文字內容，完成呼叫 `getWikipediaArticle()` 函式功能：

```
(以上省略...)
  articleTitle = arguments.join(' ');
}

print('Looking up articles about "$articleTitle". Please wait.');
```

// `print('Here you go!');` 將此行註解

// 呼叫 `getWikipediaArticle()` 函式

```
var articleContent = await getWikipediaArticle(articleTitle);

// print('Current article title: $articleTitle'); 將此行註解

```

```
print(articleContent);
```

(以下省略...)

- 因為 `getWikipediaArticle()` 函式是非同步函式，所以使用 `await` 呼叫 `getWikipediaArticle(articleTitle)`。
 - `print(articleContent)` 將列出原生 JSON 格式的資料內容。
- Task5: 更新 `main()` 函式，呼叫 `searchWikipedia()` 函式
 - 更新 `main()` 函式內，`search` 名稱改成 `wikipedia` 字眼

```
(以上省略...)
// 原本的 search 關鍵字，改成 wikipeda
// } else if (arguments.first == 'search') {

} else if (arguments.first == 'wikipedia') {
(以下省略...)
```

- Task6: 執行該 app
 - 在 Terminal 視窗中，執行下列指令：

```
dart run bin/httpex.dart wikipedia "Dart_(programming_language)"
```

- 將會看到如下訊息內容：

```
Looking up articles about "Dart_(programming_language)". Please wait.
{
  "type": "standard",
  "title": "Dart (programming language)",
  "displaytitle": "<span class=\"mw-page-title-main\">Dart (programming la
  "namespace": {
    "id": 0,
    "text": ""
  }

  // 中間省略

}
```

- 在 Terminal 視窗中，再執行下列指令：

```
dart run bin/httpex.dart wikipedia
```

- 將會看到如下訊息內容：

```
Please provide an article title.
Flutter_(software)
Looking up articles about "Flutter_(software)". Please wait.
{
  "type": "standard",
```

[Back to top](#)

```
"title": "Flutter (software)",  
"displaytitle": "<span class=\"mw-page-title-main\">Flutter (software)</span>",  
"namespace": {  
  "id": 0,  
  "text": ""  
}  
  
// 中間省略...  
  
}
```

- 重要備註：本程式重點在於：如何利用 HTTP GET 功能，向外部站台發出取得資料的 API 請求。

利用包裝與函式庫來組織 Dart 程式

- 本章節需要使用 dartex2 專案內的程式碼，在進行本章節之前，請盡力完成上一章節的範例內容。
- 本章節重點在於：如何架構一個完善的 Dart 專案
- 本章節工作重點：重構 dartex2 專案，使其接受指令列參數，進行程式運作。
- Task1: 建立 command_runner 軟體套件包
 - 切換目錄到 dartex2 專案目錄內。
 - 執行 dart 建立軟體套件包指令：

```
dart create -t package command_runner
```

- 應該會看見下列結果：

```
Creating command_runner using template package...
```

```
.gitignore  
analysis_options.yaml  
CHANGELOG.md  
pubspec.yaml  
README.md  
example\command_runner_example.dart  
lib\command_runner.dart  
lib\src\command_runner_base.dart  
test\command_runner_test.dart
```

```
Running pub get...  
Resolving dependencies...  
Downloading packages...  
Changed 48 dependencies!
```

```
Created project command_runner in command_runner! In order to get started,
```

```
cd command_runner  
dart run example/command_runner_example.dart
```

- 此時，應該有 `command_runner` 資料夾以及相關檔案目錄結構。
- Task2: 執行 `CommandRunner` 類別
 - 開啟 `command_runner/lib/command_runner.dart` 檔案，將程式碼改成下列內容：

```
/// Support for doing something awesome.
///
/// More dartdocs go here.
library;

export 'src/command_runner_base.dart';

// TODO: Export any libraries intended for clients of this package.
```

- `library;` 表示這個檔案為一函式庫檔案，定義邊界與公開且可重複使用的 Dart 程式。
- `export 'src/command_runner_base.dart';` 為一重要的宣告，表示讓其它的程式可以呼叫 `command_runner` 函式庫。
 - 如果少了本行程式，則在本檔案內的類別以及函式，將成為私有的，不利於公開給其他程式呼叫。
- 開啟 `command_runner/lib/src/command_runner_base.dart` 檔案。
- 將一些佔位符號去除，修成程式內容成如下所列：

```
class CommandRunner {
  // 給予參數，以便執行命令列應用程式
  Future<void> run(List<String> input) async {
    print('CommandRunner received input: $input');
  }
}
```

- `CommandRunner` 是類別名稱，`class` 則是字義類別的關鍵字。
- `run` 是函式，用於接收參數指令之後，在螢幕上輸出被輸入的文字。
- `Future` 是一個傳回型態，指出本方法可能執行非同步操作，但不回傳數值。
- Task3: 建立與 `command_runner` 相關連結
 - 開啟 `httpex/pubspec.yaml` 檔案。
 - 在 `httpex/pubspec.yaml` 檔案內，`name` 參數的下方加入一行程式：

```
name: httpex
public_to: none
(下方省略...)
```

- 在 `httpex/pubspec.yaml` 檔案內，於 `dependencies` 參數下方加入一行程式：

```
dependencies:
  http: ^1.3.0 # 維持原來的設定值
  command_runner:
    path: ../command_runner # 指出 command_runner 軟體封包的目錄位置
```

- Task4: 匯入使用 `command_runner` 軟體封包
 - 開啟 `httpex/bin/httpex.dart` 檔案
 - 在檔案開頭處，加入下列程式碼，引用 `command_runner` 軟體封包：

```
import 'package:command_runner/command_runner.dart';
```

- 重構 main() 函式內程式內容，移除舊有的程式，並且引用 CommandRunner 類別：

```
import 'dart:io';
import 'package:http/http.dart' as http;
import 'package:command_runner/command_runner.dart';

// 將 main() 函式修改成 async 非同步運作方式
void main(List<String> arguments) async {

  // 建立 CommandRunner 實例物件
  var runner = CommandRunner();

  // 呼叫 runner 實例內的 run() 函式
  await runner.run(arguments);

}
(以下函式刪除....)
// void searchWikipedia(List<String>? arguments) async { /* ... existing logic ... */ }
// void printUsage() { /* ... existing logic ... */ }
// Future<String> getWikipediaArticle(String articleTitle) async { /* ... existing logic ... */ }
```

- 因為 runner.run() 將回傳一個 Future 型態物件，所以 main() 函式內，必須使用 await 方式等待其回傳內容。
- var runner = CommandRunner(); 是 Dart 物件的建立方式。
 - 重要觀念：物件實例必須要由類別產生。
- printUsage、searchWikipedia、getWikipediaArticle 等函式，將移至 command_runner 內實作，故先移除。
- Task5: 執行應用程式
 - 切換目錄至 httpex 專案資料夾內
 - 執行下列程式指令：

```
dart run bin/httpex.dart wikipedia Computer_programming
```

- 應該可看見下列回應的訊息：

```
CommandRunner received input: [wikipedia, Computer_programming]
```

- 本章重要觀點：
 - 主程式內容應該要力求精簡。
 - 運用結構化模組方式，架構整個應用程式的功能。
 - 每個模組內，儘量以類別方式區分功能。
 - 每個類別程式內容，包含函式功能，均要與類別相關。
 - 程式命名需要能一眼即可辨識其功能。

參考文獻

- [Dart 官方教學手冊](#)

Generated by [aglio](#) on 12 Apr 2026